Ulster University

JavaScript

Functions

ulster.ac.uk

Functions

- Quite often we need to perform a similar action in many places of the script.
- For example, we need to show a nice-looking message when a visitor logs in, logs out and maybe somewhere else.
- Functions are the main “building blocks” of the program. They allow the code to be called many times without repetition.
- We’ve already seen examples of built-in functions, like:
  - alert(message)
  - prompt(message, default)
- But we can create functions of our own as well.

Function Declaration

- To create a function we can use a *function declaration*.

```
function showMessage() {
  alert( 'Hello everyone!' );
}
```

1. The function keyword goes first, then goes the *name of the function.*
2. Then a list of *parameters* between the parentheses (comma-separated, empty in the example above).
3. finally the code of the function, also named “the function body”, between curly braces.

```
function name(parameters) {
  ...body...
}
```

Function Calling

- Our new function can be called by its name: showMessage().

```
function showMessage() {
  alert( 'Hello everyone!' );
}

showMessage();
showMessage();
```

- The call showMessage() executes the code of the function. Here we will see the message two times.

- This example clearly demonstrates one of the main purposes of functions: to avoid code duplication.

- If we ever need to change the message or the way it is shown, it’s enough to modify the code in one place: the function which outputs it.

Functions – Local Variables

- A variable declared inside a function is only visible inside that function.

```
function showMessage() {
  let message = "Hello, I'm JavaScript!"; // local variable

  alert( message );
}

showMessage(); // Hello, I'm JavaScript!

alert( message ); // <-- Error! The variable is local to the function
```

Functions – Outer Variables

- A function can access an outer variable as well, for example:

```
let userName = 'John';

function showMessage() {
  let message = 'Hello, ' + userName;
  alert(message);
}

showMessage(); // Hello, John
```

- The function has full access to the outer variable. It can modify it as well.

Functions – Outer Variables

- For instance:

```
let userName = 'John';

function showMessage() {
  userName = "Bob"; // (1) changed the outer variable

  let message = 'Hello, ' + userName;
  alert(message);
}

alert( userName ); // John before the function call

showMessage();

alert( userName ); // Bob, the value was modified by the function
```

Functions – Outer Variables

- The outer variable is only used if there's no local one.
- If a same-named variable is declared inside the function then it *shadows* the outer one.
- For instance, in the code below the function uses the local userName. The outer one is ignored:

```
let userName = 'John';

function showMessage() {
  let userName = "Bob"; // declare a local variable

  let message = 'Hello, ' + userName; // Bob
  alert(message);
}

// the function will create and use its own userName
showMessage();
```

Functions - Parameters

- We can pass arbitrary data to functions using parameters (also called *function arguments*) .
- The example below has two parameters, from and text.

```
function showMessage(from, text) { // arguments: from, text
  alert(from + ': ' + text);
}

showMessage('Ann', 'Hello!'); // Ann: Hello! (*)
showMessage('Ann', "What's up?"); // Ann: What's up? (**)
```

Functions - Parameters

- When the function is called in lines (*) and (**), the given values are copied to local variables from and text. Then the function uses them.

```
function showMessage(from, text) { // arguments: from, text
  alert(from + ': ' + text);
}

showMessage('Ann', 'Hello!'); // Ann: Hello! (*)
showMessage('Ann', "What's up?"); // Ann: What's up? (**)
```

Functions - Parameters

- We have a variable from and pass it to the function.
- Please note: the function changes from, but the change is not seen outside, because a function always gets a copy of the value:

```
function showMessage(from, text) {

  from = '*' + from + '*'; // make "from" look nicer

  alert( from + ': ' + text );
}

let from = "Ann";

showMessage(from, "Hello"); // *Ann*: Hello
```

Functions - Default Values

- If a parameter is not provided, then its value becomes undefined.
- For instance, the aforementioned function showMessage(from, text) can be called with a single argument.

```
showMessage("Ann");
```

- That’s not an error. Such a call would output "*Ann*: undefined". There’s no text, so it’s assumed that text === undefined.

Functions - Default Values

- If we want to use a “default” text in this case, then we can specify it after =:

```
function showMessage(from, text = "no text given") {
  alert( from + ": " + text );
}

showMessage("Ann"); // Ann: no text given
```

- Now if the text parameter is not passed, it will get the value "no text given"

Functions - Default Values

- Here "no text given" is a string, but it can be a more complex expression, which is only evaluated and assigned if the parameter is missing. So, this is also possible:

```
function showMessage(from, text = anotherFunction()) {
  // anotherFunction() only executed if no text given
  // its result becomes the value of text
}
```

- In JavaScript, a default parameter is evaluated every time the function is called without the respective parameter.

- In the example above, anotherFunction() is called every time showMessage() is called without the text parameter.

Functions – Alternative Default Parameters

- Sometimes it makes sense to set default values for parameters not in the function declaration, but at a later stage, during its execution.

- To check for an omitted parameter, we can compare it with undefined:

```
function showMessage(text) {
  if (text === undefined) {
    text = 'empty message';
  }

  alert(text);
}

showMessage(); // empty message
```

Functions – Alternative Default Parameters

- Or we could use the || operator:

```
// if text parameter is omitted or "" is passed, set it to 'empty'
function showMessage(text) {
  text = text || 'empty';
  ...
}
```

- Modern JavaScript engines support the nullish coalescing operator ??, it's better when falsy values, such as 0, are considered regular:

```
// if there's no "count" parameter, show "unknown"
function showCount(count) {
  alert(count ?? "unknown");
}

showCount(0); // 0
showCount(null); // unknown
showCount(); // unknown
```

Returning a Value

- A function can return a value back into the calling code as the result.
- The simplest example would be a function that sums two values:

```
function sum(a, b) {
  return a + b;
}

let result = sum(1, 2);
alert( result ); // 3
```

Returning a Value

- The directive return can be in any place of the function. When the execution reaches it, the function stops, and the value is returned to the calling code.

- There may be many occurrences of return in a single function. For instance:

```
function checkAge(age) {
  if (age >= 18) {
    return true;
  } else {
    return confirm('Do you have permission from your parents?');
  }
}

let age = prompt('How old are you?', 18);

if ( checkAge(age) ) {
  alert( 'Access granted' );
} else {
  alert( 'Access denied' );
}
```

Returning a Value

- It is possible to use return without a value. That causes the function to exit immediately.

```
function showMovie(age) {
  if ( !checkAge(age) ) {
    return;
  }

  alert( "Showing you the movie" ); // (*)
  // ...
}
```

- In the code above, if checkAge(age) returns false, then showMovie won’t proceed to the alert.

Returning a Value

- If a function does not return a value, it is the same as if it returns undefined:

```
function doNothing() { /* empty */ }

alert( doNothing() === undefined ); // true
```

- An empty return is also the same as return undefined:

```
function doNothing() {
  return;
}

alert( doNothing() === undefined ); // true
```

Returning a Value

- **Never add a newline between return and the value!**
- For a long expression in return, it might be tempting to put it on a separate line, like this:

```
return
 (some + long + expression + or + whatever * f(a) + f(b))
```

- That doesn't work, because JavaScript assumes a semicolon after return. That'll work the same as:

```
return;
 (some + long + expression + or + whatever * f(a) + f(b))
```

Returning a Value

```
return;
 (some + long + expression + or + whatever * f(a) + f(b))
```

- So, it effectively becomes an empty return.
- If we want the returned expression to wrap across multiple lines, we should start it at the same line as return. Or at least put the opening parentheses there as follows:

```
return (
  some + long + expression
  + or +
  whatever * f(a) + f(b)
  )
```

- And it will work just as we expect it to.

Function Naming

- Functions are actions.
- So their name is usually a verb. It should be brief, as accurate as possible and describe what the function does, so that someone reading the code gets an indication of what the function does.
- It is a widespread practice to start a function with a verbal prefix which vaguely describes the action.
- There must be an agreement within the team on the meaning of the prefixes.

Function Naming

- For instance, functions that start with "show" usually show something.

- Function starting with…
  - "get…" – return a value,
  - "calc…" – calculate something,
  - "create…" – create something,
  - "check…" – check something and return a Boolean, *etc.*

```
showMessage(..)     // shows a message
getAge(..)          // returns the age (gets it somehow)
calcSum(..)         // calculates a sum and returns the result
createForm(..)      // creates a form (and usually returns it)
checkPermission(..) // checks a permission, returns true/false
```

One Function – One Action

- A function should do exactly what is suggested by its name, no more.
- Two independent actions usually deserve two functions, even if they are usually called together (in that case we can make a third function that calls those two).
- A few examples of breaking this rule:
  - getAge – would be bad if it shows an alert with the age (should only get).
  - createForm – would be bad if it modifies the document, adding a form to it (should only create it and return).
  - checkPermission – would be bad if it displays the access granted/denied message (should only perform the check and return the result).

One Function – One Action

- These examples assume common meanings of prefixes.
- You and your team are free to agree on other meanings, but usually they're not much different.
- In any case, you should have a firm understanding of what a prefix means, what a prefixed function can and cannot do.
- All same-prefixed functions should obey the rules. And the team should share the knowledge.

Functions == Comments

- Functions should be short and do exactly one thing. If that thing is big, maybe it’s worth it to split the function into a few smaller functions.
- Sometimes following this rule may not be that easy, but it’s definitely a good thing.
- A separate function is not only easier to test and debug – its very existence is a great comment.

Functions == Comments

- For instance, compare the two function showPrimes(n) below.
- Each one outputs prime numbers up to n.
- The first variant uses a label:

```
function showPrimes(n) {
  nextPrime: for (let i = 2; i < n; i++) {

    for (let j = 2; j < i; j++) {
      if (i % j == 0) continue nextPrime;
    }

    alert( i ); // a prime
  }
}
```

Functions == Comments

- The second variant uses an additional function isPrime(n) to test for primality:

```
function showPrimes(n) {

  for (let i = 2; i < n; i++) {
    if (!isPrime(i)) continue;

    alert(i);  // a prime
  }
}

function isPrime(n) {
  for (let i = 2; i < n; i++) {
    if ( n % i == 0) return false;
  }
  return true;
}
```

Functions == Comments

- The second variant is easier to understand, isn’t it?
- Instead of the code piece we see a name of the action (isPrime).
- Sometimes people refer to such code as *self-describing*.
- So, functions can be created even if we don’t intend to reuse them. They structure the code and make it readable.

Function expressions

Ulster University

Function Expressions

- The syntax that we used before is called a *Function Declaration*:

```
function sayHi() {
  alert( "Hello" );
}
```

- There is another syntax for creating a function that is called a *Function Expression*.

```
let sayHi = function() {
  alert( "Hello" );
};
```

- Here, the function is created and assigned to the variable explicitly, like any other value. No matter how the function is defined, it’s just a value stored in the variable sayHi.

Function Expressions

- The meaning of these code samples is the same: "create a function and put it into the variable sayHi".

- We can even print out that value using alert:

```
function sayHi() {
  alert( "Hello" );
}

alert( sayHi ); // shows the function code
```

- Please note that the last line does not run the function, because there are no parentheses after sayHi.

- There are programming languages where any mention of a function name causes its execution, but JavaScript is not like that.

Function Expressions

- In JavaScript, a function is a value, so we can deal with it as a value. The code previously shows its string representation, which is the source code.
- Surely, a function is a special value, in the sense that we can call it like sayHi().
- But it’s still a value. So we can work with it like with other kinds of values.

Function Expressions

- We can copy a function to another variable:

```
function sayHi() {   // (1) create
  alert( "Hello" );
}

let func = sayHi;    // (2) copy

func(); // Hello     // (3) run the copy (it works)!
sayHi(); // Hello    //     this still works too (why wouldn't it)
```

- Here's what happens above in detail:
  - The Function Declaration (1) creates the function and puts it into the variable named sayHi.
  - Line (2) copies it into the variable func. Please note again: there are no parentheses after sayHi. If there were, then func = sayHi() would write *the result of the call* sayHi() into func, not *the function* sayHi itself.
  - Now the function can be called as both sayHi() and func().

Function Expressions

- Note that we could also have used a Function Expression to declare sayHi, in the first line:

```
let sayHi = function() {
  alert( "Hello" );
};

let func = sayHi;
// ...
```

- Everything would work the same.

Why Semi-Colon?

- You might wonder, why does Function Expression have a semicolon ; at the end, but Function Declaration does not:

```
function sayHi() {
  // ...
}

let sayHi = function() {
  // ...
};
```

- There's no need for ; at the end of code blocks and syntax structures that use them like if { ... }, for { }, function f { } etc.

- A Function Expression is used inside the statement: let sayHi = ...;, as a value. It's not a code block, but rather an assignment. The semicolon ; is recommended at the end of statements, no matter what the value is.

Callback Functions

- Let's look at more examples of passing functions as values and using function expressions.
- We'll write a function ask(question, yes, no) with three parameters:
  - Question - Text of the question.
  - Yes - Function to run if the answer is "Yes".
  - No - Function to run if the answer is "No".
- The function should ask the question and, depending on the user's answer, call yes() or no()

Callback Functions

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}

function showOk() {
  alert( "You agreed." );
}

function showCancel() {
  alert( "You canceled the execution." );
}

// usage: functions showOk, showCancel are passed as arguments to ask
ask("Do you agree?", showOk, showCancel);
```

- In practice, such functions are quite useful. The major difference between a real-life ask and the example above is that real-life functions use more complex ways to interact with the user than a simple confirm.

Callback Functions

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}

function showOk() {
  alert( "You agreed." );
}

function showCancel() {
  alert( "You canceled the execution." );
}

// usage: functions showOk, showCancel are passed as arguments to ask
ask("Do you agree?", showOk, showCancel);
```

- **The arguments showOk and showCancel of ask are called *callback functions* or just *callbacks*.**
- The idea is that we pass a function and expect it to be “called back” later if necessary. In our case, showOk becomes the callback for “yes” answer, and showCancel for “no” answer.

Function Declaration vs. Function Expression

- Let's formulate the key differences between Function Declarations and Expressions.

- First, the syntax: how to differentiate between them in the code.

```
// Function Declaration
function sum(a, b) {
  return a + b;
}
```

```
// Function Expression
let sum = function(a, b) {
  return a + b;
};
```

- *Function Declaration:* a function, declared as a separate statement, in the main code flow.

- *Function Expression:* a function, created inside an expression or inside another syntax construct. Here, the function is created at the right side of the "assignment expression" =:

Function Declaration vs. Function Expression

- The more subtle difference is *when* a function is created by the JavaScript engine.
- **A Function Expression is created when the execution reaches it and is usable only from that moment.**
- Once the execution flow passes to the right side of the assignment let sum = function… – here we go, the function is created and can be used (assigned, called, etc.) from now on.
- Function Declarations are different.

Function Declaration vs. Function Expression

- **A Function Declaration can be called earlier than it is defined.**
- For example, a global Function Declaration is visible in the whole script, no matter where it is.
- That’s due to internal algorithms. When JavaScript prepares to run the script, it first looks for global Function Declarations in it and creates the functions. We can think of it as an “initialization stage”.
- And after all Function Declarations are processed, the code is executed. So it has access to these functions.

Function Declaration vs. Function Expression

- For example, this works:

```
sayHi("John"); // Hello, John

function sayHi(name) {
  alert( `Hello, ${name}` );
}
```

- The Function Declaration sayHi is created when JavaScript is preparing to start the script and is visible everywhere in it.

- …If it were a Function Expression, then it wouldn't work:

```
sayHi("John"); // error!

let sayHi = function(name) {
  alert( `Hello, ${name}` );
};
```

Block Scope

- Another special feature of Function Declarations is their block scope.
- **In strict mode, when a Function Declaration is within a code block, it's visible everywhere inside that block. But not outside of it.**
- For instance, let's imagine that we need to declare a function welcome() depending on the age variable that we get during runtime. And then we plan to use it some time later.

Block Scope

```
let age = prompt("What is your age?", 18);

// conditionally declare a function
if (age < 18) {

  function welcome() {
    alert("Hello!");
  }

} else {

  function welcome() {
    alert("Greetings!");
  }

}

// ...use it later
welcome(); // Error: welcome is not defined
```

- If we use Function Declaration, it won't work as intended.
- That's because a Function Declaration is only visible inside the code block in which it resides.

Block Scope

```
if (age < 18) {
  welcome();                // \   (runs)
                            //  |
  function welcome() {      //  |
    alert("Hello!");        //  |  Function Declaration is available
  }                         //  |  everywhere in the block where it's declared
                            //  |
  welcome();                // /   (runs)

} else {

  function welcome() {
    alert("Greetings!");
  }
}

// Here we're out of curly braces,
// so we can not see Function Declarations made inside of them.

welcome(); // Error: welcome is not defined
```

Summary

- Functions are values. They can be assigned, copied or declared in any place of the code.
- If the function is declared as a separate statement in the main code flow, that's called a "Function Declaration".
- If the function is created as a part of an expression, it's called a "Function Expression".
- Function Declarations are processed before the code block is executed. They are visible everywhere in the block.
- Function Expressions are created when the execution flow reaches them.

Summary

- In most cases when we need to declare a function, a Function Declaration is preferable, because it is visible prior to the declaration itself.
- That gives us more flexibility in code organisation and is usually more readable.